DS605 · Fundamentals of Machine Learning

Lecture Notes 18-19

Classification

Logistic Regression, Multiclass Classification, and k-Nearest Neighbors
Running example throughout: predicting ShopKart delivery outcomes.

Overview

Lectures 15–17 dealt with **regression**: predicting a continuous target such as order value or delivery time. In all of those models, the target y could be any real number, and the squared error was a sensible way to measure a mistake.

Starting with Lecture 18, the target is no longer a number — it is a **category**. Will a ShopKart order arrive **On Time** or be **delayed**? Later, a third outcome is added: **Failed**. A model that outputs an arbitrary real number cannot answer a question like this directly, so this pair of lectures builds two very different ways of handling categorical targets:

- **Logistic Regression** (and its multiclass generalization, **softmax regression**) — a parametric model that learns a fixed set of coefficients, in the same spirit as Linear Regression, but with a different output stage and a different loss function.
- **k-Nearest Neighbors (kNN)** — a non-parametric model that fits no coefficients at all, and instead classifies a new point by looking at which labeled points sit closest to it.

By the end, we compare the two families directly and set up the next topic: dimensionality reduction, which becomes necessary once kNN's notion of "nearby" starts to break down in high dimensions.

Part 1 — Logistic Regression (Lecture 18)

1.1 The target is no longer a real number

In regression, the target is $y \in \mathbb{R}$ (any real value) and the loss is squared error. In **binary classification**, the target is $y \in \{0, 1\}$ — a class label, coded here as y = 1 for "On Time" and y = 0 for "Delayed." Squared error no longer carries the same meaning once the target is a label rather than a measurement.

1.2 Why not just reuse Linear Regression?

A natural first instinct is to fit an ordinary Linear Regression directly to the 0/1 labels, using delivery distance as the feature. Doing this on the real ShopKart data yields a line with intercept 1.138 and slope −0.0693, which predicts values below 0 and above 1 for some deliveries. There are three precise reasons this approach fails, and it is worth being exact about them:

1. **Unbounded predictions.** The fitted line can predict values like −0.3 or 1.4. A probability must be between 0 and 1, so these outputs are not valid probabilities.
2. **The wrong distribution for the label.** A binary label follows a **Bernoulli** distribution, not a Gaussian one. Its conditional mean is itself a probability: $E[Y \mid X = x] = P(Y = 1 \mid X = x)$. A linear function of x has no way to guarantee that this mean stays inside [0, 1].
3. **Variance that isn't constant.** For a binary target, $\text{Var}(Y \mid X=x) = p(x)\cdot(1 - p(x))$, which changes with x. This directly breaks the constant-variance assumption that ordinary Linear Regression relies on.

(Note: it is *not* quite right to say the problem is "no natural threshold logic" — a linear regression output actually can be thresholded at 0.5. The real problems are the three above: the output isn't a valid probability, the label isn't Gaussian, and the variance isn't constant.)

Logistic Regression fixes all three problems at once: it models a *transformed* probability (the log-odds, introduced below) and fits that transformation with a likelihood built specifically for binary outcomes — not with squared error.

1.3 The sigmoid function

The fix is a function that squashes any real-valued score into the open interval (0, 1). This is the **sigmoid function**:

$$\sigma(z) = 1 / (1 + e^{-z})$$

As $z \to +\infty$, $\sigma(z) \to 1$. As $z \to -\infty$, $\sigma(z) \to 0$. At $z = 0$, $\sigma(0) = 0.5$ exactly.

Logistic Regression assembles a model in three steps, and the only genuinely new ingredient is the second one:

1. **Linear score:** $z = \beta_0 + \beta_1 x_1 + \ldots + \beta_n x_n$ — exactly the same linear predictor used in Linear Regression.
2. **Squash:** $p = \sigma(z)$ — map the score into a valid probability.
3. **Decide:** compare p to a threshold (often 0.5) to output a class label.

Everything about *fitting* β still starts from a linear combination of features; only the output stage is new.

1.4 Odds and log-odds

The **odds** of an event with probability p compare the chance it happens to the chance it doesn't:

$$\text{odds}(p) = p / (1 - p)$$

If p = 0.8, odds = 0.8/0.2 = 4 ("4 to 1 in favor"). If p = 0.5, odds = 1 (an even chance). Odds live in (0, ∞), which is already an improvement over a raw probability for some purposes — but it still isn't symmetric around "no preference," which sits at 1, not 0.

Taking the log of the odds gives the **log-odds**, also called the **logit**:

$$\text{logit}(p) = \log\left(p / (1 - p) \right)$$

Log-odds now range over all of $(-\infty, \infty)$, symmetric around 0 (which corresponds to $p = 0.5$) — exactly the range a linear predictor z already covers. This is what makes the sigmoid/logit pairing work.

Sigmoid and logit are inverse functions, not the same function — a distinction worth keeping straight:

	Input	Output	Direction
sigmoid $\sigma(z)$	score $z \in (-\infty, \infty)$	probability $p \in (0, 1)$	score → probability
logit(p)	probability $p \in (0, 1)$	score $z \in (-\infty, \infty)$	probability → score

$\text{logit}(\sigma(z)) = z$ and $\sigma(\text{logit}(p)) = p$: applying one after the other returns you to where you started.

1.5 Where the name "Logistic Regression" comes from

Starting with $p = \sigma(z)$, where $z = \beta^\top x$, a brief derivation shows what the model is really doing. From $p = 1/(1+e^{-z})$, it follows that $1 - p = e^{-z}/(1+e^{-z})$, and dividing the two expressions cancels the shared denominator, leaving:

$$p / (1 - p) = e^z$$

Taking the natural log of both sides:

$$\log\left(p / (1-p) \right) = z = \beta_0 + \beta_1 x_1 + \ldots + \beta_n x_n$$

This equation is what "Logistic Regression" means: the model assumes the conditional log-odds of the positive class are a **linear function of the features**. It is important not to describe this as "an ordinary linear regression fit to observed log-odds" — individual labels are 0 or 1, and the logit of 0 or 1 is $-\infty$ or $+\infty$, so there is nothing to regress against directly. Instead, β is estimated by **maximum likelihood**, using a likelihood built for binary outcomes (derived in section 1.8 below). The model never predicts y directly; it predicts z, a linear function of x, and every statement about probability is a transformation applied on top of that.

1.6 Fitting the model and reading the coefficients

Fitting a real single-feature model (distance only) to the ShopKart data gives $\beta_0 = 3.642$ and $\beta_1 = -0.397$. The negative coefficient means farther deliveries are less likely to be on time.

Reading the sign. In this single-feature model, increasing distance by one unit changes the log-odds of being on time by exactly β_1: $\Delta\text{logit}(p) = \beta_1 \cdot \Delta x$. A negative β_1 means the log-odds — and therefore the probability — of on-time delivery decreases as distance grows. The sign has a direct, unambiguous reading in log-odds space, even though its effect on p itself is nonlinear.

A more useful reading: the odds ratio. Exponentiating a coefficient converts a log-odds change into a multiplicative change in the odds themselves:

$$\text{odds ratio} = e^{\beta_j}$$

e^{β_j} is the factor by which the odds of Y=1 are multiplied when x_j increases by one unit, holding the other features fixed. Here, $e^{-0.397} \approx 0.672$, so each additional kilometer of delivery distance multiplies the odds of on-time delivery by about 0.672 — a reduction of roughly 32.8%. One caution: when features are **standardized** before fitting (as in the multi-feature models later in this lecture), the coefficient — and its odds ratio — describes the effect of a *one-standard-deviation* increase, not a one-unit increase in the original measurement.

1.7 The threshold and the decision boundary

The model's raw output is a probability $p \in (0, 1)$. Turning that into a class label requires a threshold τ, and $\tau = 0.5$ is only the default — not something the model demands:

$$\acute{y} = 1 \text{ if } p \geq \tau, \text{ else } 0$$

Raising τ makes the model more conservative about predicting "On Time"; lowering it makes that prediction easier to trigger. Neither choice is forced by the math — it is a modeling decision, made explicit in code as tau = 0.7; y_pred = (probs >= tau).astype(int).

This generalizes cleanly. Since $p \geq \tau$ if and only if $z \geq \log(\tau/(1-\tau))$, the threshold decision is always a comparison against a shifted version of the same linear boundary:

$$p \geq \tau \;\Leftrightarrow\; z \geq \log\left(\tau / (1 - \tau)\right)$$

At $\tau = 0.5$, $\log(\tau/(1-\tau)) = 0$, recovering the familiar $z = 0$ cutoff. A different τ only shifts which side of the *same* linear boundary counts as positive; it does not change the boundary's shape. With two features, the boundary where $p = 0.5$ (equivalently, $z = 0$) is a straight line; with more features, it becomes a hyperplane.

1.8 Deriving the loss function: binary cross-entropy

Now that there is a probability p and a true label $y \in \{0, 1\}$, nothing stops us from computing $(p - y)^2$ and calling it a loss — but this would be a mistake, for two reasons. First, squared error is the statistically right loss *when errors are Gaussian*, a story that made sense for a continuous target, not a 0/1 label. Second, composed with the sigmoid, squared error produces a loss surface with flat regions where gradients vanish, slowing learning exactly where the model is most wrong. Instead, the loss is *derived* from a probabilistic model of the label itself.

Step 1 — model one label as a Bernoulli trial. Treat y as a coin flip with bias p (the model's predicted probability):

$$P(y \mid x) = p^{y}(1-p)^{1-y}$$

If $y = 1$, this is p. If $y = 0$, this is $1-p$. One expression correctly captures "the probability of the

outcome that actually happened," for either label.

Step 2 — the likelihood of the whole training set. Assuming the m training examples are independent, the probability of observing all of their labels is the product of the individual Bernoulli probabilities:

$$L(\beta) = \prod_{i=1..m} p_i^{y_i}(1-p_i)^{1-y_i}$$

Maximum likelihood means choosing β to make the observed labels as probable as possible under the model. Each p_i depends on β through $p_i = \sigma(\beta^\top x_i)$.

Step 3 — log, negate, average. Taking the log turns this product into a *sum*, which is much easier to differentiate, while preserving the same maximizer, since log is strictly increasing. As a bonus, it also avoids the numerical underflow that a product of many small probabilities would cause. Negating the result (to turn "maximize" into "minimize") and averaging over m examples gives **binary cross-entropy**:

$$J(\beta) = -(1/m) \sum_{i=1..m} [\, y_i \log(p_i) + (1-y_i) \log(1-p_i) \,]$$

This loss is *derived* from a Bernoulli likelihood, not declared by fiat. Maximizing the likelihood is exactly the same as minimizing J(β).

1.9 Fitting β, and an important caveat about regularization

J(β) has no closed-form solution, the way Ordinary Least Squares did — it must be minimized iteratively. Gradient Descent is one valid way to do this, and it is useful for building intuition, but it is *not* scikit-learn's default. Several solver families exist: **lbfgs** (the scikit-learn default, a fast and robust quasi-Newton method), **liblinear** (coordinate-descent based, common for smaller or sparse datasets), and **newton-cg**, **sag**, **saga**, and **newton-cholesky** (additional families with different scaling and penalty support; newton-cholesky is a newer, efficient option when there are many rows relative to features).

Differentiating J(β) gives a gradient with a strikingly familiar shape:

$$\nabla J(\beta) = (1/m)\, X^\top (p - y)$$

Compare this to Linear Regression's gradient, $\nabla J(\beta) = (1/m)X^\top(\hat{y} - y)$. With p (a probability) in place of ŷ (a real-valued prediction), the two gradients have *exactly* the same form: features, transposed, times a residual vector. The update rule is the same as before: $\beta \leftarrow \beta - \alpha \nabla J(\beta)$.

An important distinction the derivation above glosses over: the J(β) derived here is the *unregularized* loss. scikit-learn's LogisticRegression applies ℓ_2 regularization *by default*, so the objective it actually optimizes also contains a penalty term, conceptually:

$$J_{reg}(\beta) = J(\beta) + \lambda \|\beta\|_2^2$$

The regularization strength is controlled by a parameter called C, which behaves like an *inverse* regularization strength: smaller C means stronger regularization (LogisticRegression(C=0.1) is strongly regularized; LogisticRegression(C=100) is weakly regularized). Some references state $C = 1/\alpha$ exactly — treat that as a loose intuition rather than a universal identity, since the precise scaling depends on the solver and loss normalization used internally. (It's also worth knowing

that LogisticRegression and SGDClassifier are separate, independently implemented estimators in scikit-learn, using different solvers and code paths, even though SGDClassifier(loss='log_loss') can approximate similar results.)

1.10 The full fit, in code — and avoiding a leakage trap

Putting the pieces together, the correct way to fit and evaluate this model uses a **pipeline**, not a manually-applied scaler:

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

features = ['DeliveryDistance_km', 'ItemsInCart']

model = make_pipeline(StandardScaler(), LogisticRegression())
model.fit(X_train[features], y_train)

p_hat = model.predict_proba(X_test[features])[:, 1]   # probabilities
y_hat = model.predict(X_test[features])                # labels at tau=0.5
```

A Pipeline fits the scaler on X_train only, then reuses that *same* fitted scaler — unchanged — to transform X_test. This is what keeps information from the test set out of preprocessing, and it is the pattern to reuse inside cross-validation later in the course. (A tempting but incorrect alternative is to fit a scaler once on the whole dataset before splitting, or to fit the test-set transform separately — both leak information and should be avoided.) The real fitted coefficients for this two-feature, standardized model are $\beta_0 = 0.404$, $\beta = [-1.533, 0.115]$.

1.11 Evaluating a classifier honestly

The confusion matrix. Before precision and recall can be defined, it helps to name the four ways a prediction can relate to the truth. At a fixed threshold τ, every prediction falls into exactly one of four cells:

	Predicted On Time	Predicted Delayed
Actual On Time	**TP** — True Positive	FN — False Negative
Actual Delayed	FP — False Positive	**TN** — True Negative

$$\text{Accuracy} = (TP + TN) / (TP + TN + FP + FN)$$

Every metric that follows — precision, recall, F1, accuracy — is just a different way of combining these four counts.

Precision and recall. *Precision* asks: of the examples predicted On Time, what fraction actually were? Precision = TP/(TP+FP). *Recall* asks: of the examples that actually were On Time, what fraction did we catch? Recall = TP/(TP+FN). On the real ShopKart data, sweeping the threshold shows the familiar trade-off:

Threshold τ	Precision	Recall
0.30	0.691	0.935
0.50 (default)	0.771	0.826
0.70	0.854	0.568

Raising τ trades recall for precision (fewer false "On Time" calls, but more missed ones); lowering it does the reverse. Which trade-off is right depends on the cost of each mistake — a modeling decision, not a mathematical one.

Choosing τ the correct way. The sweep above is run on a held-out *validation* set, not the test set — and this matters. The correct workflow is: (1) train the model on the training set; (2) choose τ using validation data or cross-validation; (3) evaluate the chosen τ exactly once, on the test set. Letting the test set guide the choice of τ quietly leaks test information into what is supposed to be a final, honest evaluation.

The F1-score. Precision and recall can be combined into a single number, their harmonic mean:

$$F_1 = 2 \cdot (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

At τ = 0.5, $F_1 \approx 0.798$. F_1 is useful when both false positives and false negatives matter and there is no clear reason to prefer one over the other; it rewards a model only when both precision *and* recall are reasonably high — but, like precision and recall themselves, it completely ignores true negatives.

Class imbalance: why accuracy can mislead. If 95% of ShopKart orders arrive on time, a model that *always* predicts "On Time" scores 95% accuracy while catching zero delayed orders. Accuracy alone cannot tell a genuinely good model apart from one that has simply learned to predict the majority class. Useful tools here include **stratified splitting** (keeping each class's proportion the same across train/validation/test, via train_test_split(..., stratify=y)), **balanced accuracy** (the average of each class's recall, so a large class can't dominate the score), the distinction between **macro** and **weighted averaging** (macro treats every class equally; weighted behaves more like plain accuracy), and class_weight='balanced' (re-weights the training loss so mistakes on rare classes count more — a reasonable first thing to try, not a complete fix).

A note on calibration. A predicted probability is only useful *as a probability* if the model is calibrated: among all orders assigned P(On Time) ≈ 0.8, roughly 80% should actually arrive on time, not merely have the highest score among similar orders. Calibration is a separate question from accuracy: a model can rank orders correctly, and score well on accuracy, precision, or recall, while its reported probabilities are still systematically too confident or too cautious.

1.12 Recap of Lecture 18

The lecture's arc, end to end: a categorical target breaks the assumptions behind Linear Regression (**Problem**) → a linear score $z = \beta^\top x$ is squashed through σ(z) into a probability (**Model**) → binary cross-entropy, derived from a Bernoulli likelihood, replaces squared error

(**Loss**) → a probability becomes a label only after choosing a threshold τ (**Decision**) → the model is checked against a majority-class baseline, a confusion matrix, and F1 or balanced accuracy when classes are imbalanced (**Evaluate**). Linear Regression predicts a continuous target; Logistic Regression uses a linear predictor to model log-odds, and thereby produces class probabilities.
ShopKart orders, in reality, don't only end up "On Time" or "Delayed" — some fail to deliver at all. With three outcomes instead of two, does everything that was built just survive the change? That question opens Part A.

Part 2 — Multiclass Logistic Regression (Lecture 19, Part A)

2.1 Two strategies for more than two classes

Every ShopKart order now resolves to one of three outcomes: OnTime, Delayed, or Failed. Everything in Lecture 18 assumed exactly two classes — one sigmoid, one probability p, one 1−p. There are two genuinely different ways to extend the model:

- **Strategy 1: combine binary classifiers.** Train several ordinary binary logistic regressions and combine their outputs, either One-vs-Rest or One-vs-One.
- **Strategy 2: softmax regression.** Generalize the sigmoid itself so the model natively outputs a probability for each of K classes at once.

2.2 One-vs-Rest and One-vs-One

One-vs-Rest (OvR) trains K separate binary classifiers. Classifier k learns to distinguish "class k" from "everything else", for example, OnTime vs. {Delayed, Failed}. At prediction time, all K classifiers run, and the class whose classifier is most confident wins. It is worth being precise about a real limitation here: each OvR classifier is trained independently, with no constraint that its outputs coexist sensibly with the others. A component classifier need not even produce a calibrated probability. "Pick the most confident one" is a reasonable heuristic, not a guarantee that the K scores sum to one coherent probability distribution, which is part of why a genuinely multiclass model is often preferred when calibrated class probabilities matter.

One-vs-One (OvO) trains one binary classifier for every *pair* of classes (OnTime vs Delayed, OnTime vs Failed, Delayed vs Failed for K=3), a total of K(K−1)/2 classifiers. Every pairwise classifier casts a vote for one of its two classes, and the class with the most votes wins.

The one comparison that holds in general is the growth rate: OvR scales as O(K), OvO as $O(K^2)$. For K=10, that's 10 classifiers versus 45; for K=50, it's 50 versus 1225. It is tempting, but incorrect, to also claim that one strategy is generally "more accurate" than the other — that depends heavily on the dataset and the base classifier.

2.3 Softmax regression

The single most common misreading of softmax is treating it like a sigmoid — a scalar curve that can be plotted as a single S-shape. Softmax is not that: it is a **vector-valued function**. K logits go in, K probabilities come out, and every output probability depends on *every* input logit: $z = (z_1, \ldots, z_K) \rightarrow \text{softmax}(z) = (p_1, \ldots, p_K)$. There is no meaningful way to draw softmax as a

single 2D curve, since its input and output both live in K dimensions at once.

Where the logits come from. Multiclass logistic regression fits a *separate* set of coefficients for every class. For K classes, each class k gets its own linear score, computed from the same input x:

$$z_k = \beta_{0k} + \beta_k^\top x, \quad k = 1, \ldots, K$$

One input x produces K linear scores $z_1, \ldots, z_K$ — one per class, each with its own β_k — and softmax turns those K scores into K probabilities that sum to 1. These z_k are the logits; they don't appear from nowhere.

The softmax formula:

$$p_k = e^{z_k} / \sum_{j=1..K} e^{z_j}, \quad \text{for each } k = 1, \ldots, K$$

The numerator exponentiates one class's logit — larger logits become disproportionately larger. The denominator sums all exponentiated logits; it is the same for every k, and is what makes the outputs sum to 1. Every p_k depends on the full vector z through this shared denominator, which is exactly why softmax must be understood as one K-in, K-out function.

A worked example. Take illustrative (not fitted) logits z = [2, 1, 0]. Then $e^2 \approx 7.39$, $e^1 \approx 2.72$, $e^0 = 1$, summing to ≈ 11.11, giving P ≈ [0.665, 0.245, 0.090]. Softmax preserves the *ordering* of the logits, and turns logit *differences* into probability *ratios*: $z_1 - z_2 = 1$ becomes $p_1/p_2 = e^1 \approx 2.72$, close to the observed 0.665/0.245 ≈ 2.71.

Softmax is shift-invariant. Adding the same constant a to every logit leaves every output probability unchanged: softmax(z + a) = softmax(z) for any scalar a. Adding a = 10 to z = [2,1,0] gives z = [12,11,10] — a completely different vector, yet the resulting probabilities are still [0.665, 0.245, 0.090]. Implementations exploit this for numerical stability: subtracting max(z) from every logit before exponentiating keeps the numbers small (avoiding overflow) without changing the answer:

$$p_k = e^{z_k - z_{max}} / \sum_j e^{z_j - z_{max}}$$

2.4 Categorical cross-entropy

Represent the true label as a one-hot vector y (all zeros except a 1 at the true class). The natural multiclass generalization of Lecture 18's Bernoulli likelihood gives, for one example:

$$J_i(\beta) = -\sum_{k=1..K} y_k \log(p_k)$$

Because exactly one entry of y is 1 and the rest are 0, every term except the true class vanishes, and this collapses to a single, interpretable quantity per example:

$$J_i(\beta) = -\log(p_{\text{true class}})$$

The loss, for one example, is just the negative log of the probability the model assigned to the correct class. Confident and correct ($p_{true} \to 1$) gives loss → 0; confident and wrong ($p_{true} \to 0$) gives loss → ∞. Averaged over the training set: $J(\beta) = -(1/m)\sum_i \log(p_{i,\text{ true class}})$.

Consistency check. Set K = 2 with classes {0, 1} and $p_1 = p$, $p_0 = 1-p$. For a true label $y \in \{0, 1\}$, categorical cross-entropy's single active term becomes $-\log(p_{true}) = -[y \log p + (1-y) \log(1-p)]$. At the level of the loss *value*, this reduces exactly to Lecture 18's binary cross-entropy. (The two parameterizations differ — softmax fits a separate score per class, sigmoid fits a single score — but the probability-level loss each one optimizes agrees exactly when K = 2.) Softmax regression is a strict generalization of logistic regression, not a different idea.

2.5 Fitting the real 3-class model, and reading accuracy honestly

Fitting a real model on distance and package weight:

```
features = ['DeliveryDistance_km', 'PackageWeight_kg']

model = make_pipeline(StandardScaler(), LogisticRegression(max_iter=2000))
model.fit(X_train[features], y_train)     # y_train has 3 classes
model.score(X_test[features], y_test)     # -> 0.667
```

The default lbfgs solver fits the multinomial model automatically for 3 or more classes (the older multi_class='multinomial' argument is deprecated; liblinear does not support this joint formulation).

A bare accuracy of 0.667 has no context on its own, so it's worth reading honestly. The test set (n=60) breaks down as OnTime 39 (65.0%), Delayed 14 (23.3%), Failed 7 (11.7%), with this confusion matrix:

Actual \ Predicted	OnTime	Delayed	Failed
OnTime	**30**	7	2
Delayed	6	**6**	2
Failed	3	0	**4**

A majority-class baseline that always predicts "OnTime" would already score 39/60 = 65.0% accuracy — without learning anything. The fitted model's 66.7% is only a small improvement over the baseline. Macro-F1 ≈ 0.58 (averaging per-class F1 of 0.77, 0.44, and 0.53 for OnTime, Delayed, and Failed) makes this clearer than accuracy does, since it weighs all three classes equally instead of letting the large OnTime class dominate.

2.6 A worked prediction, start to finish

For a real order at 12 km, 5 kg, the fitted model outputs P = [0.062, 0.617, 0.321] across [OnTime, Delayed, Failed] — predicted class: **Delayed**. The three probabilities sum to exactly 1, a guarantee OvR's independently trained scores don't automatically give you.

2.7 Three strategies, compared

	# classifiers	Native probabilities?	Typical use
One-vs-Rest	O(K)	Not guaranteed to be coherent	Simple, scales to many classes
One-vs-One	$O(K^2)$	Vote-based, not probabilistic	Can help with some base classifiers
Softmax	1 joint model	Yes, by construction	scikit-learn's default for multiclass LogisticRegression

Softmax regression extends logistic regression from two classes to K, using one linear predictor per class and a single joint probability distribution over all of them, with binary logistic regression as the special case K = 2. Every model covered so far: Linear Regression through softmax regression, fits parameters β from data, then discards the training examples. Part B asks a very different question: what if we skip fitting parameters altogether?

Part 3 — k-Nearest Neighbors (Lecture 19, Part B)

3.1 A non-parametric idea

What if a new point were classified just by asking which labeled points sit closest to it? This is **k-Nearest Neighbors (kNN)**: no β, no loss function to minimize, no parameter-estimation phase in the usual sense — the entire training set *is* the model. In three steps:

1. **Store** the entire labeled training set. There is nothing to "fit."
2. **Measure** the new point's distance to every stored point.
3. **Aggregate** the labels of the k closest points — vote for classification, average for regression.

3.2 Measuring distance

Euclidean distance is the familiar straight-line distance:

$$d_{\text{euclidean}}(a, b) = \sqrt{\left(\sum_{j=1..n} (a_j - b_j)^2 \right)}$$

Between (0,0) and (3,4): $\sqrt{(3^2+4^2)} = \sqrt{25} = 5$.

Manhattan distance sums distance along the axes instead of cutting diagonally:

$$d_{\text{manhattan}}(a, b) = \sum_{j=1..n} |a_j - b_j|$$

For the same two points: |3−0| + |4−0| = 7, longer than the Euclidean path. Both are special cases of the **Minkowski distance** family, $(\sum_j |a_j - b_j|^p)^{1/p}$: Manhattan is p=1, Euclidean is p=2.

Categorical and missing features need care. This lecture uses only numeric features, but real data rarely does. Euclidean distance does not directly make sense for a variable like "delivery city"; there is no natural notion of how far "Delhi" is from "Pune." One-hot encoding is a common fix, but it can distort distances, especially for high-cardinality categories. A distance to a missing coordinate is undefined, so missing values must be imputed or otherwise handled before computing distances. In general, the chosen distance metric should reflect what the features actually mean; Euclidean is a default, not a law.

3.3 Feature scaling is essential for kNN

Distance is computed directly on raw feature values. If one feature ranges over 20–60 (age, say) and another over 20,000–200,000 (income), the income differences numerically swamp the age differences, and kNN would effectively ignore age entirely — regardless of which feature actually matters. This parallels Ridge's scaling requirement from Lecture 16, but for kNN, unscaled features don't just bias the fit; they can make the distance metric meaningless outright.

3.4 kNN for regression

To predict a number, average the targets of the k nearest neighbors:

$$\hat{y}(x) = (1/k) \sum_{i \in N_k(x)} y_i$$

For a real order at 10 km with k=9, the 9 nearest neighbors' order values average to ₹504 — exactly what KNeighborsRegressor returns.

Weighted kNN regression lets closer neighbors count more, using weights='distance':

$$\hat{y}(x) = \sum_i w_i y_i / \sum_i w_i, \quad w_i = 1/d(x, x_i)$$

For the same query and neighbors, weighting by inverse distance shifts the prediction from ₹504 to ₹511, pulled toward the closest neighbors' values. A practical detail: if a query point exactly coincides with a stored point, d=0 and 1/d is undefined. Implementations handle this as a special case — typically giving an exact match full weight (or averaging over all exact matches) rather than dividing by zero.

3.5 kNN for classification

To predict a class, take a majority vote among the k nearest neighbors. For a real query point (9 km, 3 kg) with 9 nearest neighbors, 7 vote OnTime and 2 vote Delayed, so the majority vote predicts OnTime. This also gives a natural notion of class probability:

$$\hat{P}(Y=c \mid x) = (\text{number of the k neighbors from class c}) / k$$

This is what KNeighborsClassifier.predict_proba() returns under uniform weighting — here, $\hat{P}$(OnTime) = 7/9 ≈ 0.78 (a weighted analogue applies under weights='distance'). These are local, empirical proportions, not the output of a fitted probability model, and they need not be well calibrated.

On choosing k odd, and ties. For *binary* classification, choosing an odd k guarantees the vote can never end in an exact tie between the two classes. This guarantee is

binary-classification-specific: with three or more classes, an odd k can still tie (k=9 with three classes can split 3–3–3). Multiclass ties require an implementation-specific rule — there is no universal convention. In scikit-learn, even the order of the training observations can affect the result when the k-th and (k+1)-th nearest points are equally distant but carry different labels, so it is worth inspecting the library's actual behavior rather than assuming an odd k removes all ties.

3.6 Choosing k: bias, variance, and cross-validation

k is the central hyperparameter of kNN, and it controls model complexity much like λ did for Ridge and Lasso — though through a different mechanism. A small k (e.g., k=1) gives high variance: the boundary chases individual points, including noise, directly paralleling overfitting with weak regularization or high-degree polynomials. A large k gives high bias: the boundary is smoothed past the real structure in the data, the kNN analog of underfitting. Concretely: larger k means more neighborhood averaging (a smoother boundary, higher bias); larger λ means stronger coefficient shrinkage (a simpler parametric model, higher bias). The bias-variance *effect* is analogous; the *mechanism* producing it is not the same.

k is chosen in the same honest way λ was: by cross-validation. A real 5-fold GridSearchCV over k ∈ {1, 3, 5, 9, 15, 25, 41, 61} traces a U-shaped accuracy curve and finds the best k = 5:

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import GridSearchCV

knn_pipeline = Pipeline([('scaler', StandardScaler()), ('knn',
KNeighborsClassifier())])

search = GridSearchCV(knn_pipeline, {'knn__n_neighbors': [1, 3, 5, 9, 15, 25,
41, 61]}, cv=5)
search.fit(X_train, y_train)
search.best_params_   # -> {'knn__n_neighbors': 5}
```

Scaling lives *inside* the pipeline on purpose: on every cross-validation fold, StandardScaler is re-fit on that fold's training rows only, so no information from the held-out fold ever leaks into it — the same leakage risk flagged earlier for a single train/test split.

3.7 Computational characteristics

	Logistic / Softmax Regression	k-Nearest Neighbors
Fitting cost	Iterative optimization over all data, once	Mainly stores (and possibly indexes) the data — no parameter estimation
Prediction cost	One dot product per prediction — cheap	Brute-force compares against every stored point; tree-based

	Logistic / Softmax Regression	k-Nearest Neighbors
		indexes (KD-tree, ball tree) can reduce this in lower-dimensional data
Memory at prediction time	Just the fitted β	The entire training set
Interpretability	Coefficients have a direct log-odds reading	No global parameters to inspect

3.8 The curse of dimensionality

kNN's entire premise is that "nearby points" are meaningfully more similar than distant ones, and that premise gets shakier as the number of features grows. In a real Monte Carlo experiment, 40,000 random points were sampled uniformly in a unit hypercube for each dimension, and the fraction landing within distance 0.5 of the center collapses toward zero as the dimension grows. A second view of the same effect: the spread between the closest and farthest pairwise distances, relative to the average distance, also shrinks as the dimension grows. In high dimensions, *everything* is roughly equally far away. If every point is about equally distant from every other point, "nearest neighbor" stops conveying useful information. This is the **curse of dimensionality**, and it hits kNN especially directly.

3.9 Logistic Regression vs. kNN

	Logistic / Softmax Regression	k-Nearest Neighbors
Model shape assumed	A specific (linear-in-log-odds) form	None — purely local
Decision boundary	With features entering linearly, a hyperplane in feature space	Can be arbitrarily curved
Effect of feature scaling	Important when regularization is used, and often helpful for optimization	Fundamental — scale directly determines the distance
Behavior in high dimensions	Degrades gracefully with regularization	Degrades sharply — the curse of dimensionality

3.10 Closing, and what's next

kNN relies entirely on meaningful distances between points. As the number of features grows into the hundreds or thousands, those distances stop being meaningful — every point becomes roughly equally far from every other. The next lecture takes up **Dimensionality Reduction**: techniques that compress many features down to the few that actually carry signal. This can restore meaningful "nearby" neighborhoods when the discarded dimensions were mostly noise or redundancy — a help in that case, not a guarantee in general.

Summary

Two families of classifiers were built in these two lectures, each from a very different starting point. **Logistic and softmax regression** assume a specific functional form (linear in the log-odds), fit a fixed set of coefficients once by maximizing a likelihood derived from the Bernoulli/categorical distribution, and afterward make predictions cheaply from those coefficients alone. **k-Nearest Neighbors** assumes nothing about the shape of the boundary, fits no parameters at all, and instead makes every prediction by measuring distances to the stored training data, which makes feature scaling non-negotiable and leaves it vulnerable to the curse of dimensionality. Across both families, a recurring theme was *evaluating honestly*: reading accuracy against a baseline, using a confusion matrix and F1 rather than accuracy alone when classes are imbalanced, and choosing any hyperparameter (a threshold τ, or k) using validation data or cross-validation — never the test set.